Vincent Terrelonge
GitHub Link

Chapter 6

1. We change the expression() method to call the new comma() method:

```
private Expr expression() {
      return comma();
}

private Expr comma() {
      Expr expr = equality();

      while (match(COMMA)) {
            Token operator = previous();
            Expr right = equality();
            expr = new Expr.Binary(expr, operator, right);
            }

      return expr;
}
```

   To prevent the expression() method from interpreting all commas as a single argument, we add this to finishCall() by replacing `expression();`:

```
if (!check(RIGHT_PAREN)) {
      do {
      if (arguments.size() >= 8)
                  error(peek(), "arguments must be < 8");
            expression();
            arguments.add(equality());
      } while (match(COMMA));
}
```

2. The precedence of the left operator is high while the middle operator is low. The whole operator is right associative since `a ? b : c ? d : e` gets parsed as `a ? b : (c ? d : e)`.

```
private Expr expression() {
return conditional();
}

private Expr conditional() {
Expr expr = equality();

if (match(QUESTION)) {
```

```
                Expr thenBranch = expression();
                consume(COLON, "Expect ':' after conditional.");
                Expr elseBranch = conditional();
            expr = new Expr.Conditional(expr, thenBranch, elseBranch);
      }

      return expr;
      }
```

3.

```
private Expr primary() {
      if (match(FALSE)) return new Expr.Literal(false);
      if (match(TRUE)) return new Expr.Literal(true);
      if (match(NIL)) return new Expr.Literal(null);

      if (match(NUMBER, STRING))
            return new Expr.Literal(previous().literal);

      if (match(LEFT_PAREN)) {
            Expr expr = expression();
            consume(RIGHT_PAREN, "Expect ')' after expression.");
            return new Expr.Grouping(expr);
      }

      if (match(BANG_EQUAL, EQUAL_EQUAL))
            return errorProduction("Missing left operand.", this::comparison);

      if (match(GREATER, GREATER_EQUAL, LESS, LESS_EQUAL))
            return errorProduction("Missing left operand.", this::term);

      if (match(PLUS))
            return errorProduction("Missing left operand.", this::factor);

      if (match(SLASH, STAR))
            return errorProduction("Missing left operand.", this::unary);

      throw error(peek(), "Expect expression.");
}

private Expr errorProduction(String message, Supplier<Expr> rule) {
      error(previous(), message);
      rule.get();
      return new Expr.Literal(null);
}
```

Chapter 7

1. I would definitely expand lox to be able to compare different objects of the same type. For example, the .equals() method in Java allows checking if objects have the same data stored. It would also be useful to compare different strings to see if one is greater or less than the other based on alphabetical order.

2. ```
   case PLUS:
         if (left instanceof String || right instanceof String)
               return stringify(left) + stringify(right);

         if (left instanceof Double && right instanceof Double)
               return (double)left + (double)right;
   ```

3. Dividing a number by zero yields either ±inf or NaN. Other languages usually result in a runtime error so that the user knows something is wrong or sometimes languages will explicitly say there was a divide by zero error.

   ```
   case SLASH:
         checkNumberOperands(expr.operator, left, right);

         if ((double)right == 0)
               throw new RuntimeError(expr.operator, "Divide by zero
         error.");

         return (double)left / (double)right;
   ```
   ```